International Journal of Science and Research (IJSR)
ISSN: 2319-7064
ResearchGate Impact Factor (2018): 0.28 | SJIF (2018): 7.426

Strategies for Mitigating Flaky Tests in Automated Environments

Rohit Khankhoje

Independent Researcher, Avon, Indiana, USA
Email: *rohit.khankhoje[at]gmail.com*

Abstract: ***Automated testing, a crucial aspect of software development, plays an essential role in assuring the dependability and effectiveness of applications. Nevertheless, the presence of flaky tests, which exhibit unpredictable outcomes, presents a significant challenge that undermines the stability and credibility of automated testing suits. This article delves into the key issue of flaky tests in automated environments, offering a comprehensive analysis of their causes, ramifications, and strategies for mitigation. Through a combination of scholarly literature review and empirical investigation, the study identifies crucial factors that contribute to test flakiness, such as inconsistencies in the environment, concurrency problems, and inadequate test design. Additionally, it explores a variety of mitigation strategies, including advanced detection methods, improved patterns in test design, and techniques for ensuring environmental stability. The article presents a case study and experimental evaluations to demonstrate the effectiveness of these strategies in real - world scenarios. The findings disclose that a combination of proactive test design, robust management of the environment, and continual monitoring can significantly decrease the prevalence of flaky tests. This research contributes to the field of software testing by providing actionable strategies for practitioners to enhance the dependability of their automated testing procedures, as well as by establishing a foundation for future research in this vital area of software quality assurance.***

Keywords: Automated Testing, Flaky Test, Software Testing, Quality Assurance

1. Introduction

In the contemporary realm of software development, test automation has emerged as an indispensable tool, empowering teams to efficiently validate the functionality, performance, and reliability of software. Nevertheless, amid the advancements in this field, the industry encounters a substantial and persistent challenge: the prevalence of unreliable tests. These unreliable tests are characterized by inconsistent outcomes, fluctuating between successful and unsuccessful states without any modifications to the code or testing environment. This phenomenon not only jeopardizes the credibility of testing results but also impedes the efficiency of development processes.

The ramifications of unreliable tests extend beyond mere inconveniences. In environments that rely on automated testing, where dependability and consistency are of utmost importance, unreliable tests can trigger a series of adverse consequences. They erode trust in the accuracy of test results, cause delays in delivery schedules, and necessitate additional resources for troubleshooting and rectification. Furthermore, the existence of unreliable tests can obscure genuine issues, potentially leading to the release of faulty software. This situation is particularly critical in Continuous Integration/Continuous Deployment (CI/CD) pipelines, where automated tests play a crucial role in the delivery process.

Acknowledging the critical nature of this problem, this study aims to analyze the perplexing nature of unreliable tests in automated environments. The research paper aims to investigate and clarify the underlying causes of test unreliability, encompassing environmental instabilities and inherent flaws in test design and scripting. A significant emphasis of the study is to propose and assess various strategic interventions and best practices that can be implemented to effectively detect, manage, and mitigate unreliable tests.

FlakyTest - In the realm of test automation, the term "flaky test" refers to a type of test that demonstrates inconsistent outcomes, alternating between successful and unsuccessful results without any modifications to the underlying code or environment. This lack of predictability renders flaky tests problematic, as they are unable to reliably indicate whether a software application is operating correctly. The sources of flakiness can vary, encompassing timing discrepancies, dependencies on external systems, non - deterministic behaviors, or inadequately constructed test scripts. Flaky tests erode confidence in automated testing procedures, complicate the process of debugging, and may result in failures being overlooked or unnecessary delays in the software development life cycle. The resolution of flaky tests is of utmost importance in order to uphold the effectiveness and integrity of the automated testing process.

The extent of this paper encompasses a wide range of software applications, including various automated testing frameworks and environments. By conducting a thorough review of existing literature, analyzing case studies, and conducting empirical research, this study aims to provide a comprehensive understanding of the phenomenon of flaky tests. Its objective is to equip software testers, developers, and quality assurance professionals with practical insights and methodologies to effectively address this pervasive challenge. Through direct confrontation of the issue of flaky tests, the study aspires to make a significant contribution towards enhancing the quality, reliability, and efficiency of software development and testing processes within the realm of automated testing.

Licensed Under Creative Commons Attribution CC BY

2. Literature Review

Current methodologies employed in automated testing environments to deal with unstable tests expose a significant disparity between the identification and resolution of these unforeseeable issues. While conventional approaches, such as rerunning failed tests or isolating unstable ones, provide temporary remedies, they frequently fail to address the underlying causes of instability. This issue is worsened by an excessive dependence on manual debugging procedures, which are time - consuming and not easily scalable, particularly in the context of extensive and intricate test suites. Moreover, the absence of sophisticated tools and frameworks specifically designed to systematically detect, analyze, and rectify unstable tests further exacerbates this gap. These tools are imperative for effectively predicting potential instability and offering comprehensive analysis, yet their development and integration into existing testing environments have been restricted. Another crucial gap pertains to the management of test environment stability; fluctuations in the testing environment can often result in unstable tests, but this aspect is frequently neglected in favor of solely focusing on the test scripts themselves. Furthermore, current methods encounter difficulties with scalability and efficiency, as they are not always equipped to handle the expanding size and complexity of test suites in rapidly evolving development cycles. This situation necessitates a transition towards more advanced, data - driven, and AI - based approaches that can provide predictive insights, automate the management of instability, and ensure more robust and dependable test automation frameworks.

Table 1: Traditional way to handle flaky test

Traditional way	Approach	Limitation
Rerunning Failed Tests	Commonly, teams rerun tests that fail to determine if they are flaky.	This method is time - consuming and doesn't address the root cause of flakiness. It may also lead to ignoring genuine failures, assuming them to be flaky.
Isolation of Flaky Tests	Flaky tests are identified and isolated from the main test suite.	While this keeps the main suite stable, it often results in a backlog of flaky tests that get less attention over time, potentially hiding underlying issues.
Increased Timeouts and Delays	Increasing timeouts or adding delays to handle synchronization and timing issues.	This can lead to increased test execution times and does not guarantee the resolution of flakiness, especially if the root cause is unrelated to timing.
Manual Inspection and Debugging	Manual investigation of flaky tests to identify and fix issues.	This process is labor - intensive and not scalable. It also relies heavily on the expertise and experience of the QA team.
Logging and Monitoring	Implementing extensive logging to track down when and why a test behaves flakily.	While useful for diagnosis, extensive logging can become overwhelming and may not always lead to a straightforward solution.
Test Quarantining	Temporarily removing flaky tests from the test suite until they are fixed.	Quarantining can lead to prolonged neglect of flaky tests, and essential tests might be out of action for extended periods.
Static Analysis Tools	Using static analysis tools to identify potentially flaky tests based on code patterns.	These tools might not detect all flaky tests, especially those caused by external factors like network issues or database dependencies.

The examination of existing literature concerning the characteristics and origins of unreliable tests uncovers a multifaceted problem deeply ingrained in automated testing environments. Unreliable tests are defined as those that exhibit uncertain outcomes, inconsistently passing or failing without any modifications to the code or external conditions. The literature identifies multiple primary causes: inconsistencies in the environment, such as variations in test execution environments or external dependencies; timing issues, including race conditions and insufficient wait times in asynchronous operations; non - deterministic order of test execution; and deficiencies in test design, such as lack of isolation or dependence on external states. Studies also highlight the use of external services and network dependencies as significant contributors to the unreliability. Additionally, the literature emphasizes the role of poorly maintained test data and configuration issues in contributing to this unpredictability. This body of work underscores the intricate, often interconnected nature of factors leading to unreliable tests, necessitating a comprehensive and nuanced approach to their identification, management, and resolution.

Identifying Flaky Tests

Detecting flaky tests in automated testing environments is an essential and pivotal undertaking that necessitates the utilization of a combination of strategic approaches and specific criteria. Provided below is a comprehensive overview of the principal criteria and strategies employed for this purpose. By correlating these strategies with the particular criteria, teams have the ability to embrace a more precise methodology in identifying and subsequently mitigating the presence of inconsistency within their collections of tests. This methodical association guarantees that every facet of inconsistency is tackled with the utmost efficient strategy, resulting in a more resilient and dependable automated testing procedure. Furthermore, in conjunction with these explicit correspondences, certain tactics offer more comprehensive assistance and have the potential to be efficiently implemented across various standards.

Licensed Under Creative Commons Attribution CC BY

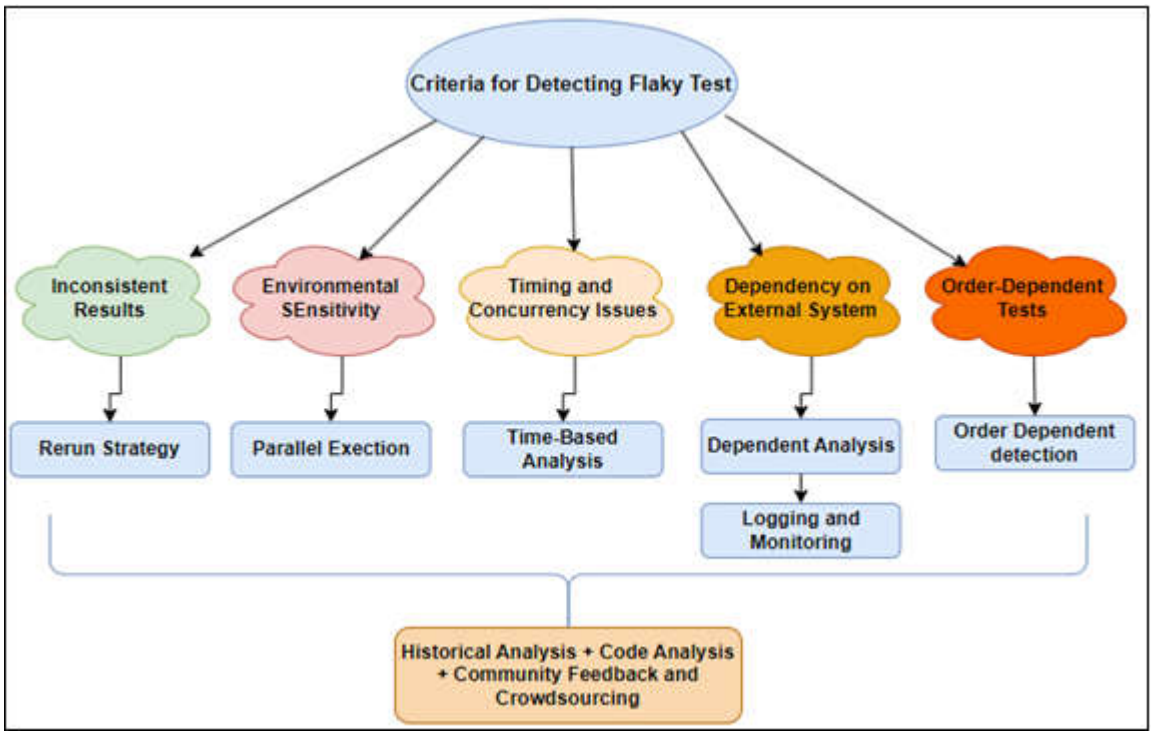


Figure 1: Criteria vs Strategy

Criteria for Detecting Flaky Tests:

1) **Inconsistent Results**: The identification of tests that exhibit non - deterministic outcomes, namely intermittent success and failure, under identical conditions.
2) **Environmental Sensitivity**: Tests that solely fail in specific environments or under particular configurations may serve as an indication of flakiness.
3) **Timing and Concurrency Issues**: Tests that fail due to timing problems or concurrency issues, such as race conditions, are often characterized by their flaky nature.
4) **Dependency on External Systems**: The occurrence of flakiness can be attributed to tests that rely on external systems or services that lack consistency or are occasionally unavailable.
5) **Order - Dependent Tests**: Tests that produce varied outcomes depending on the sequence in which they are executed can be indicative of flakiness.

Strategies for Detecting Flaky Tests:

1) **Rerun Strategy**: The automated repetition of failed tests multiple times to ascertain the presence of inconsistent results. Furthermore, the implementation of a predefined threshold for reruns is recommended to strike a balance between detection and resource utilization.
2) **Historical Analysis**: The scrutiny of historical test data to identify recurrent patterns of inconsistency. Additionally, the utilization of machine learning algorithms to forecast potential flaky tests based on past behavior is advised.
3) **Time - Based Analysis**: The systematic monitoring of tests for failures that exhibit a correlation with specific time periods or occur subsequent to updates made to the testing environment.
4) **Parallel Execution in Varying Environments**: The execution of tests in parallel across diverse environments or configurations to effectively identify environment - sensitive flakiness.
5) **Code Analysis:** The application of static code analysis techniques to uncover code smells or patterns that commonly give rise to flakiness, such as improper setup/teardown, shared state, or unmocked external calls.
6) **Dependency Analysis**: The thorough examination of tests to detect dependencies on external systems, and the flagging of tests that display a high reliance on these systems.
7) **Order - Dependence Detection**: The execution of tests in a randomized order to expose any potential issues related to order dependency.
8) **Logging and Monitoring**: The implementation of comprehensive logging mechanisms for tests, and the continuous monitoring of logs to identify patterns or anomalies that suggest the presence of flakiness.
9) **Community Feedback and Crowdsourcing**: The utilization of feedback provided by developers and testers who regularly work with the tests. In addition, crowdsourced insights can often accurately identify flaky tests based on human experience and intuition.

By diligently applying these well - defined criteria and strategies, teams can effectively and efficiently detect flaky tests, thereby taking the initial step towards addressing and mitigating their impact on the software development lifecycle. This proactive approach is of paramount importance for upholding the integrity and dependability of automated testing suites.

Mitigation Strategies

The examination of the application and efficacy of every tactic to alleviate unreliable tests necessitates an assessment

Licensed Under Creative Commons Attribution CC BY

of how these approaches are put into practice and an evaluation of their influence on the dependability and effectiveness of the tests.

Table 2: Mitigation Strategy and Effectiveness

Strategy	Implementation	Effectiveness
Improving Test Isolation	Test cases are redesigned to be self - contained, with mock objects or stubs used for external dependencies.	Enhances test reliability by removing external factors, but requires careful management of mock objects to ensure they accurately represent real - world scenarios.
Enhancing Test Environment Stability	Standardized environments are created using containerization tools.	Offers high consistency across test runs, reducing environmental flakiness, but may introduce complexity in managing containerized environments.
Addressing Timing and Concurrency Issues	Incorporate explicit waits and synchronization mechanisms in tests.	Reduces flakiness due to timing issues, but may increase test complexity and execution time.
Data Management	Use separate databases or data sets for each test or test run.	Prevents data - related flakiness; however, it requires additional setup to manage isolated data environments.
Rerun Flaky Tests with Analysis	Automatically rerun failed tests and analyze the outcomes.	Useful for immediate identification of flaky tests, but doesn't address the root cause and could lead to ignoring real issues.
Code Quality and Design Patterns	Regular refactoring and adoption of patterns like Page Object Model in UI tests.	Improves maintainability and readability of tests, but requires ongoing effort and adherence to best practices.
Use of Advanced Tools and Technologies	Employ specialized tools for flaky test detection and predictive analytics.	Can significantly enhance the detection process; however, the accuracy depends on the sophistication of the tools.
Comprehensive Logging and Monitoring	Detailed logging for each test run and monitoring of test trends.	Facilitates in - depth analysis of flaky tests, though it may generate large volumes of data to sift through.
Community and Team Collaboration	Regular meetings, knowledge - sharing sessions, and collective ownership of test quality.	Fosters a proactive approach to test maintenance, but relies heavily on team culture and collaboration.
Regular Audits and Reviews	Periodic reviews of the test suite and code reviews focusing on test scripts.	Helps in early identification and rectification of potential flakiness but requires dedicated time and resources.

The efficacy of these strategies in addressing the issue of unreliable tests is contingent upon the specific circumstances in which they are employed, encompassing factors such as the project's inherent characteristics, the testing environment, and the team's dedication to ensuring quality assurance. Employing a combination of multiple strategies typically yields optimal outcomes in diminishing the occurrence of test flakiness.

3. Discussion

Evaluating the efficacy of diverse mitigation techniques for unreliable tests necessitates a nuanced comprehension of how each approach addresses specific facets of instability in automated testing. Strategies that isolate tests and concentrate on ensuring each test is independent prove highly effective in mitigating interdependencies and interactions that contribute to unreliability, especially in the case of unit and integration tests. However, their effectiveness may be hampered by the intricate nature of implementing extensive mocking and stubbing.

Approaches that foster environmental stability, which involve standardizing and controlling the testing environment, significantly diminish external factors that contribute to unreliability. This strategy is particularly critical for end - to - end tests as it ensures consistency across test runs. Nevertheless, the effectiveness of this strategy is somewhat tempered by the resource - intensive process of setting up and maintaining controlled environments.

The Rerun Strategy, which entails repeatedly executing failed tests to identify instances of unreliability, provides a straight forward and expeditious means of detecting flaky tests. Although it effectively identifies non - deterministic tests, it fails to address the underlying causes, thus serving as a short - term resolution.

Code Quality and Design Patterns play a pivotal role in enhancing the maintainability and resilience of test scripts. By refactoring tests and employing patterns such as the Page Object Model in UI testing, this strategy significantly reduces the likelihood of encountering flakiness. However, its effectiveness is limited by the continuous effort and profound comprehension of design principles that it demands.

Advanced Tools and Technologies, such as specialized software for detecting flaky tests and AI - based predictive models, offer efficient and often automated means of identifying unreliable tests. Nevertheless, their effectiveness may be constrained by integration challenges with existing testing environments and the level of sophistication of the tools.

Comprehensive Logging effectively facilitates in - depth analysis of test executions, aiding in the diagnosis of the underlying causes of unreliability. However, the effectiveness of this strategy may be impeded by the copious volume of data generated, which can become burdensome to analyze.

Team Collaboration and Regular Audits ensure a proactive approach to maintaining the health of tests. By fostering a culture centered on quality and conducting ongoing health checks of the tests, these strategies prove effective. However, their effectiveness is contingent upon team

Licensed Under Creative Commons Attribution CC BY

dynamics, active participation, and the allocation of dedicated time for reviews.

In summary, the effectiveness of each mitigation strategy varies depending on the types of tests, available resources, and the specific challenges posed by unreliability. Often, a combination of these strategies, tailored to the unique context of the project, yields the most comprehensive and effective solution in mitigating flaky tests.

4. Conclusion

The issue of unreliable tests in automated testing environments poses a significant obstacle to the dependability and efficiency of the software development lifecycle. This research paper has delved into various strategies for alleviating the impact of these unpredictable tests, offering a comprehensive approach that encompasses enhancing test isolation, stabilizing the testing environment, addressing timing concerns, managing data effectively, utilizing advanced tools, and fostering collaborative teamwork.

The effectiveness of these strategies lies in their holistic application. By integrating practices that isolate tests and standardized testing environments, teams can significantly reduce external factors that contribute to test instability. Implementing flexible timing strategies and robust data management practices further ensures that tests are less vulnerable to timing and data - related inconsistencies. The selective rerun of tests, along with comprehensive logging and monitoring, provides valuable insights into the nature of instability, enabling teams to identify and address underlying issues more effectively.

These points open an avenue for future work -

- Flakiness primarily arises from the interactions among various components of the system, the testing infrastructure, and uncontrollable external factors. To address flakiness, forthcoming research endeavors can utilize monitoring and log analysis to propose techniques that aid practitioners.
- One crucial measure to prevent flaky tests is the establishment of straightforward testing guidelines, which include recommendations on test size, external resources, and assertion thresholds. In the future, studies can reduce the manual effort required to enforce these guidelines by offering static analysis tools and incorporating code review processes.
- To effectively expose and reproduce flaky tests, future work can take advantage of variability - aware reruns and fuzzy testing. Such techniques have the potential to automate the current manual test validations performed by practitioners.
- Considering the frequency of flaky tests and the cost associated with mitigating them, practitioners rely on the flake rate to adapt their strategies. In the assessment of flaky tests and the development of automated solutions, future endeavors should take into account this indicator.
- In some cases, practitioners mistakenly categorize buggy and nondeterministic features as flaky tests, leading them to disregard these issues as false alerts. Further research should explore the consequences of such confusions.
- Due to the challenges involved in reproducing and debugging flaky tests, the task of fixing them is rarely accomplished by practitioners. Therefore, future work should concentrate on providing tools that aid in identifying the root cause and reproducing flaky tests.

It is evident that there is no universal solution to the problem of unreliable tests. Each strategy possesses its own unique advantages and must be tailored to the specific context of the project and testing environment. The key to success lies in a balanced and adaptable approach, continuously adjusting and refining strategies based on feedback and changing project dynamics.

In conclusion, effectively mitigating the issue of unreliable tests requires a multifaceted approach that combines technical solutions, process enhancements, and a strong emphasis on collaborative teamwork and continuous learning. As the field of software testing continues to evolve, it is crucial to stay updated on new challenges and solutions in managing unreliable tests to ensure the delivery of high - quality software products.

References

[1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests, ” in Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering, vol.16 - 21 - November - 2014. Association for Computing Machinery, nov 2014, pp.643–653.

[2] R. K. Saha, L. Zhang, S. Khurshid, and D. E. Perry, “REPiR: An Information Retrieval based Approach for Regression Test Prioritization, ” FSE ’14, Hongkong, 2014.

[3] H. Jiang, X. Li, Z. Yang, and J. Xuan, “What causes my test alarm? automatic cause analysis for test alarms in system and integration testing, ” in Proceedings of the 39th International Conference on Software Engineering, ser. ICSE ’17. IEEE Press, 2017, p.712–723. [Online]. Available: https: //doi. org/10.1109/ICSE.2017.71

[4] A. Micco, John & Memon, “Gtac 2016: How flaky tests in continuous integration - youtube, ” https: //www.youtube. com/watch?v=CrzpkF1 - VsA, December 2016

[5] J. Listfield, “Google testing blog: Where do our flaky tests come from?” https: //testing. googleblog. com/2017/04/ where - do - our - flaky - tests - come - from. html, April 2017

[6] F. J. Shull, J. C. Carver, S. Vegas, and N. Juristo, “The role of replications in Empirical Software Engineering, ” Empirical Software Engineering, vol.13, no.2, pp.211–218, 2008.

[7] O. S. Gomez, N. Juristo, and S. Vegas, “Understanding replication of experiments in software engineering: A classification, ” Information and Software Technology, vol.56, no.8, pp.1033–1048, 2014.

Licensed Under Creative Commons Attribution CC BY